

Trabalho Prático de Sistemas Operativos

2025/2026



Grupo 21:

A108361 – António Pedro Aguiar Amaral de Sousa Barroso

A110770 – Eduardo Gabriel Esteves Adão

A111262 – Tomás Teles Coelho

Índice:

1. Introdução
2. Arquitetura do *controller/runner*
3. Comunicação entre processos
4. Estado interno do *controller* e escalonamento
5. Execução de comandos, pipes e redirecionamentos
6. Consulta, log e encerramento
7. Avaliação experimental
8. Limitações
9. Conclusão

1. Introdução

O presente relatório descreve a solução desenvolvida para o trabalho prático de Sistemas Operativos (SO), cujo objetivo é implementar um sistema de gestão e escalonamento de comandos através de dois programas principais: *controller* e *runner*.

O *runner* é o programa utilizado pelos utilizadores para submeter comandos, consultar o estado do sistema ou pedir o encerramento do serviço. O *controller* é responsável por receber os pedidos dos vários *runners*, manter o estado global do sistema, escalonar os comandos recebidos e autorizar a sua execução quando houver capacidade disponível.

A solução implementada separa claramente o escalonamento da execução. O *controller* não executa comandos diretamente, apenas decide quando cada comando pode iniciar. A execução fica a cargo do respetivo *runner*, que só avança depois de receber autorização. Esta arquitetura permite controlar a concorrência, aplicar políticas de escalonamento e manter informação sobre os comandos em execução e em espera.

A comunicação entre processos foi implementada com pipes com nome (FIFOs), usando um FIFO principal para envio de pedidos ao *controller* e um FIFO privado para cada *runner* receber respostas. Além da execução simples de comandos, a solução também suporta redirecionamentos de entrada, saída e erro, pipelines, paralelismo configurável e encerramento controlado do sistema.

2. Arquitetura do *controller/runner*

O *controller* é o processo central do sistema. É responsável por criar o FIFO principal, receber pedidos dos vários *runners*, manter informação sobre os comandos em execução e em espera, aplicar a política de escalonamento escolhida e autorizar a execução de comandos quando existir a capacidade disponível. O *controller* também é responsável por responder a pedidos de consulta, registar no ficheiro de log os comandos terminados e tratar o encerramento controlado do sistema.

O *runner* é o programa executado pelos utilizadores. Em modo de execução, cria um FIFO privado, envia ao *controller* um pedido com o identificador do utilizador, o identificador do comando e a string do comando a executar. Depois disso, fica bloqueado à espera da autorização do *controller*. Só após receber essa autorização é que executa o comando. Quando a execução termina, envia uma nova mensagem ao *controller* a indicar que o comando terminou.

Esta arquitetura garante que o *controller* não executa comandos diretamente. A sua função é apenas coordenar e escalonar. A execução dos comandos fica a cargo dos respetivos *runners*. Esta separação facilita o controlo do número máximo de comandos em paralelo e permite implementar diferentes políticas de escalonamento sem alterar a forma como os comandos são executados.

Em termos de fluxo normal de execução, o processo ocorre da seguinte forma:

1. O utilizador invoca o *runner* com a opção `-e`
2. O *runner* cria o seu FIFO privado
3. O *runner* envia um pedido de execução para o FIFO principal do *controller*

4. O *controller* recebe o pedido e decide se o comando pode iniciar imediatamente ou se deve ficar em espera
5. Quando o comando puder iniciar, o *controller* envia uma autorização pelo FIFO privado do *runner*
6. O *runner* executa o comando
7. No fim da execução, o *runner* envia uma mensagem de conclusão ao *controller*
8. O *controller* atualiza o seu estado interno, escreve no log e tenta lançar novos comandos que estejam em espera

Para além deste fluxo, o *runner* pode também ser utilizado para consultar o estado do sistema com a opção -c ou para pedir o encerramento do *controller* com a opção -s. Em ambos os casos, o pedido é enviado ao *controller*, que processa a informação e responde através do FIFO privado do *runner*.

3. Comunicação entre processos

A comunicação entre os *runners* e o *controller* foi implementada com pipes com nome (FIFOs). Esta escolha permite a comunicação entre processos independentes, que não têm relação direta de parentesco, o que se adequa ao modelo do trabalho, uma vez que vários *runners* podem ser executados em terminais diferentes.

A solução usa um FIFO principal, chamado *Fifo_Server*, criado pelo *controller* no arranque. Todos os *runners* enviam os seus pedidos para este FIFO. Cada pedido é enviado através da estrutura *Msg* que contém o PID do *runner*, o tipo de pedido, o identificador do utilizador, o identificador do comando e a string do comando a executar.

Para evitar ambiguidades nas respostas, cada *runner* cria também um FIFO privado antes de contactar o *controller*. O nome desse FIFO é construído com base no PID do processo, no formato *fifo_<pid>*. Assim, quando o *controller* precisa de responder a um pedido, sabe exatamente para que FIFO deve escrever.

Foram definidos 4 tipos de mensagens:

- *TYPE_EXEC*: pedido de execução do comando
- *TYPE_STATUS*: pedido de consulta do estado
- *TYPE_STOP*: pedido de encerramento do *controller*
- *TYPE_DONE*: notificação do fim de execução

No caso de um pedido de execução, o *runner* envia uma mensagem tipo *TYPE_EXEC* e fica à espera de uma resposta no seu FIFO privado. Se o *controller* tiver capacidade para iniciar o comando, ou quando chegar a vez desse comando na fila de espera, envia *RESP_AUTHORIZED*. Só depois disso é que o *runner* executa o comando. Se o sistema já estiver em processo de encerramento, o pedido é rejeitado com *RESP_REJECTED*.

A comunicação para pedidos de consulta e de encerramento segue o mesmo princípio. O pedido é enviado pelo FIFO principal e a resposta é devolvida pelo FIFO privado do *runner*. No caso da consulta a resposta contém a lista de comandos em execução e em espera. No caso do encerramento, o *controller* responde apenas quando já não existirem comandos ativos ou em espera.

Esta organização evita que vários *runners* leiam respostas que não lhes pertencem e simplifica a associação entre cada pedido e a respetiva resposta. Ao mesmo tempo, mantém o *controller* como ponto central de coordenação do sistema.

4. Estado interno do *controller* e do escalonamento

O *controller* mantém o estado global do sistema através de duas estruturas principais: uma lista de comandos em execução e uma fila de comandos em espera. Esta separação permite distinguir os comandos que já foram autorizados dos comandos que ainda aguardam autorização.

A lista de comandos em execução guarda, para cada comando ativo, o PID do *runner*, o identificador do utilizador, o identificador do comando, a string do comando e o instante em que o pedido foi recebido pelo *controller*. Esta informação é usada tanto para responder a pedidos de consulta como para calcular a duração total do comando quando este termina.

A fila de espera contém os comandos que ainda não foram autorizados. Cada entrada guarda a mensagem original enviada pelo *runner* e o instante de submissão. Quando termina um comando em execução, o *controller* verifica se existe capacidade disponível e tenta lançar novos comandos que estejam em espera.

O número máximo de comandos em execução em simultâneo é definido no arranque do *controller* através do argumento `parallel-commands`. Para controlar esse limite, o *controller* mantém um contador de comandos ativos. Quando um comando é autorizado, o contador é incrementado. Quando o *runner* informa que terminou a execução, o contador é decrementado.

Foram implementadas duas políticas de escalonamento: FIFO e round-robin. Na política FIFO, os comandos são autorizados pela ordem de chegada. Esta política é simples e previsível, mas pode fazer com que os utilizadores com poucos comandos fiquem à espera se outro utilizador tiver submetido vários comandos antes (starvation).

Na política round-robin, o *controller* tenta escolher o primeiro comando em espera pertencente a um utilizador diferente do último utilizador autorizado. Esta abordagem procura melhorar a justiça entre utilizadores, evitando que um único utilizador domine o sistema com muitos comandos consecutivos. Se todos os comandos em espera pertencerem ao mesmo utilizador, a política comporta-se como FIFO. A variável que guarda o último utilizador escolhido é atualizada sempre que um comando é autorizado, mesmo que esse comando comece imediatamente sem passar pela fila de espera. Esta decisão é importante para manter a consistência da política round-robin em cenários com múltiplos *runners* concorrentes.

Quando é feita uma consulta com *runner -c*, os comandos em espera são apresentados pela ordem em que seriam escalonados. No caso da política FIFO, essa ordem corresponde diretamente à ordem física da fila. No caso do round-robin, o *controller* simula a ordem de escalonamento numa cópia da fila, sem alterar o estado real do sistema.

5. Execução de comandos, pipes e redirecionamentos

A execução dos comandos é feita pelo *runner* após receber autorização do *controller*. Esta decisão mantém a separação entre escalonamento e execução: o *controller* decide quando um comando pode iniciar, mas não o executa diretamente. Depois de receber autorização, o *runner* processa a string do comando submetido pelo utilizador. O comando é dividido em tokens, reconhecendo argumentos, operadores de redirecionamento e operadores de pipeline. Foram suportados os operadores: `>`, `<`, `2>` e `|`.

O operador `>` redireciona o standard output para um ficheiro.

O operador `<` redireciona o standard input a partir de um ficheiro.

O operador `2>` redireciona o standard error.

O operador `|` permite ligar o output de um comando ao input do comando seguinte, formando uma pipeline.

Cada comando simples dentro de uma pipeline é representado internamente pelos seus argumentos e pelos possíveis ficheiros de redirecionamento. Depois de analisada a string do comando, o *runner* cria os pipes necessários e lança um processo filho para cada comando da pipeline.

Em cada processo filho são feitas as ligações necessárias com `dup2`. Se o comando não for o primeiro da pipeline, o seu input é ligado ao pipe anterior. Se o comando não for o último, o seu output é ligado ao pipe seguinte. Depois são aplicados os redirecionamentos definidos pelo utilizador. Por fim, o comando é executado com `execvp`.

A utilização de `execvp` permite executar diretamente o programa indicado pelo utilizador, sem recorrer a `system()` nem a uma shell intermédia. Assim, a execução fica mais controlada.

O processo pai fecha os descritores de ficheiro que já não são necessários e espera pela terminação dos processos filhos. No caso de uma pipeline, o estado final considerado corresponde ao estado do último comando executado.

A implementação também trata comandos mal formados, como pipelines sem comando antes ou depois do operador `|`, e redirecionamentos sem ficheiro associado. Nestes casos, o runner termina a execução do comando de forma controlada, evitando deixar processos bloqueados.

6. Consulta, log e encerramento controlado

Para além da execução de comandos, o runner suporta pedidos de consulta e de encerramento do sistema. Estes pedidos são enviados ao controller através do FIFO principal, usando o mesmo mecanismo de comunicação usado para os pedidos de execução.

A consulta do estado do sistema é feita com a opção `-c`:

```
./bin/runner -c
```

Neste modo, o runner cria o seu FIFO privado, envia uma mensagem TYPE_STATUS ao controller e aguarda a resposta. O processamento da consulta é feito pelo controller, que envia ao runner a lista de comandos em execução e a lista de comandos em espera.

A resposta é apresentada em duas secções: Executing e Scheduled. A primeira contém os comandos que já foram autorizados e ainda não terminaram. A segunda contém os comandos que aguardam autorização, apresentados pela ordem em que serão escalonados.

Para evitar que uma consulta bloqueie o ciclo principal do controller, a resposta ao pedido de estado é feita por um processo filho. Assim, o processo principal continua disponível para receber novos pedidos de execução, conclusão ou encerramento.

Quando um comando termina, o runner envia ao controller uma mensagem TYPE_DONE. O controller remove esse comando da lista de execução, calcula a duração total desde a submissão até ao fim e escreve a informação no ficheiro log.txt.

Cada entrada do log contém o identificador do utilizador, o identificador do comando, a string do comando executado e a duração total em milissegundos. Este valor inclui tanto o tempo em espera como o tempo efetivo de execução, uma vez que é calculado a partir do instante em que o pedido chegou ao controller.

O encerramento do sistema é pedido com a opção -s:
./bin/runner -s

Quando recebe uma mensagem TYPE_STOP, o *controller* não termina imediatamente se ainda existirem comandos ativos ou em espera. Em vez disso, marca que foi pedido encerramento e deixa os comandos já aceites terminarem normalmente.

Depois de recebido o pedido de encerramento, novos pedidos de execução são rejeitados explicitamente com RESP_REJECTED. Esta decisão evita que novos *runners* fiquem bloqueados à espera de autorização num sistema que já está a terminar.

O *controller* só encerra quando não existem comandos ativos nem comandos em espera. Antes de terminar, envia RESP_SHUTDOWN_DONE ao *runner* que pediu o encerramento, fecha o FIFO principal, remove-o do sistema de ficheiros e liberta as estruturas usadas internamente.

7. Avaliação experimental

Para validar a solução foram desenvolvidos scripts em bash que automatizam os testes funcionais e a avaliação experimental. Os testes foram executados em ambiente Linux, após uma compilação limpa do projeto com make clean e make.

Foram criados 4 scripts:

- testa_funcionamento.sh
- avalia_parallelismo.sh
- compara_politicas.sh
- corre_todos.sh

O script testa_funcionamento.sh valida a correção funcional do sistema. O script avalia_parallelismo.sh mede o impacto do valor de parallel-commands no tempo total de execução. O script compara_politicas.sh compara o comportamento das políticas fifo e

rr. Por fim, o script `corre_todos.sh` executa os três scripts anteriores de forma sequencial.

7.1 Testes funcionais

O script `testa_funcionamento.sh` executa um conjunto de testes para verificar se as principais funcionalidades estão corretas. Foram testadas a compilação, a execução de comandos simples, os redirecionamentos `>`, `<` e `2>`, as pipelines com `|`, a consulta do estado, o registo no log, a execução paralela, o comportamento sequencial quando `parallel-commands` é igual a 1, o encerramento controlado e a rejeição de comandos após o pedido de shutdown.

Na execução realizada, todos os testes funcionais passaram.

Total de Testes: 18; Passaram 18; Falharam: 0;

Estes testes confirmam que o sistema se comporta corretamente em cenários normais e também em situações críticas, como ausência do *controller*, submissão de comandos após pedido de encerramento e execução concorrente de vários *runners*.

7.2 Impacto do paralelismo

Para avaliar o impacto do número máximo de comandos paralelos, foram executados 8 comandos `sleep 2` com a política `fifo`, variando o valor de `parallel-commands`.

Política	parallel-commands	Nº comandos	Comando	Tempo total (ms)
fifo	1	8	sleep 2	16 098
fifo	2	8	sleep 2	8 065
fifo	4	8	sleep 2	4 049
fifo	8	8	sleep 2	2 045

Os resultados mostram uma redução clara do tempo total à medida que o número máximo de comandos paralelos aumenta. Com `parallel-commands = 1`, os 8 comandos são executados sequencialmente e o tempo total fica próximo de 16 segundos. Com `parallel-commands = 2`, o tempo desce para cerca de 8 segundos. Com 4 comandos em paralelo, o tempo fica próximo de 4 segundos, e com 8 comandos em paralelo aproxima-se dos 2 segundos.

Este comportamento confirma que o limite de paralelismo definido no arranque do *controller* está a ser respeitado e que o sistema consegue executar vários comandos em simultâneo quando existem vagas disponíveis.

7.3 Comparação das políticas de escalonamento

Para comparar as políticas `fifo` e `rr`, foi usado um cenário com `parallel-commands = 1`, em que o utilizador 1 submete três comandos `sleep 2` antes dos utilizadores 2 e 3 submeterem comandos curtos. Este cenário permite observar se os utilizadores com menos comandos ficam bloqueados atrás de um utilizador que submeteu vários comandos seguidos.

Política	User	Nome	Comando	Tempo total (ms)
fifo	1	u1_a	sleep 2	2 042
fifo	1	u1_b	sleep 2	4 050
fifo	1	u1_c	sleep 2	6 065
fifo	2	u2_a	echo user2	6 069
fifo	3	u3_a	echo user3	6 079
rr	1	u1_a	sleep 2	2 043
rr	2	u2_a	echo user2	2 048
rr	1	u1_b	sleep 2	4 060
rr	3	u3_a	echo user3	4 065
rr	1	u1_c	sleep 2	6 078

Com a política fifo, os comandos são autorizados pela ordem de chegada. Como os três primeiros comandos pertencem ao utilizador 1, os comandos dos utilizadores 2 e 3 só terminam por volta dos 6 segundos, apesar de serem comandos curtos.

Com a política rr, o *controller* tenta escolher um comando de um utilizador diferente do último autorizado. Assim, o comando do utilizador 2 termina logo após o primeiro sleep 2, por volta dos 2 segundos, e o comando do utilizador 3 termina por volta dos 4 segundos. Isto mostra que a política round-robin melhora a justiça entre utilizadores, reduzindo o tempo de espera de utilizadores que submeteram menos comandos.

7.4 Discussão dos resultados

Os testes funcionais validam as funcionalidades principais do sistema e mostram que não foram detetadas falhas nos cenários testados. Em particular, foi verificado que o *runner* não executa comandos sem autorização, que os redirecionamentos e pipelines funcionam corretamente, que o estado do sistema pode ser consultado e que o encerramento é feito de forma controlada.

A avaliação do paralelismo demonstra que o parâmetro parallel-commands tem impacto direto no tempo total de execução. Os resultados obtidos estão próximos dos valores esperados para comandos sleep 2, o que indica que o *controller* respeita o limite de comandos concorrentes.

A comparação entre políticas mostra a diferença entre uma política simples e uma política mais justa. A política fifo privilegia a ordem de chegada, enquanto a política rr tenta alternar entre utilizadores. No cenário testado, a política rr reduziu significativamente o tempo de espera dos utilizadores 2 e 3, evitando que ficassem bloqueados atrás de todos os comandos do utilizador 1.

8. Limitações

A solução implementada suporta as funcionalidades pedidas, nomeadamente a execução de comandos, redirecionamentos >, < e 2>, pipelines com |, consulta do estado, para-

lelismo configurável e políticas de escalonamento. No entanto, não se pretende implementar uma shell completa.

Também foram definidos limites fixos para alguns elementos, como o tamanho máximo da string do comando, o número máximo de argumentos, o número máximo de tokens e o número máximo de comandos numa pipeline. Estes limites simplificam a implementação e são suficientes para os testes realizados, mas poderiam ser substituídos por estruturas dinâmicas numa versão mais geral.

A política round-robin implementada é simples e baseada no último utilizador autorizado. Esta abordagem melhora a justiça face à política FIFO em cenários com vários utilizadores, mas não considera métricas mais avançadas, como tempo médio de espera, prioridades ou histórico completo de execução. Uma versão futura poderia usar filas separadas por utilizador e recolher estatísticas adicionais para tomar decisões de escalonamento mais informadas.

9. Conclusão

O trabalho desenvolvido permitiu implementar um sistema de gestão e escalonamento de comandos baseado em dois programas principais: o *controller* e o *runner*. A solução separa claramente a coordenação da execução: o *controller* mantém o estado global do sistema e decide quando cada comando pode iniciar, enquanto o *runner* executa o comando apenas depois de receber autorização.

A comunicação entre processos foi implementada através de pipes com nome, usando um FIFO principal para envio de pedidos ao *controller* e FIFOs privados para envio de respostas aos *runners*. Esta solução permitiu suportar vários *runners* concorrentes e associar corretamente cada resposta ao processo que fez o pedido.

Foram implementadas as funcionalidades principais pedidas no enunciado: submissão e execução de comandos, consulta de comandos em execução e em espera, encerramento controlado, registo persistente em log, redirecionamentos, pipelines, paralelismo configurável e políticas de escalonamento FIFO e round-robin.

A avaliação experimental confirmou o funcionamento da solução. Os testes funcionais passaram na totalidade, os resultados de paralelismo mostraram a redução esperada no tempo total de execução quando aumenta o número de comandos paralelos, e a comparação entre políticas mostrou que o round-robin melhora a justiça entre utilizadores em cenários onde um utilizador submete vários comandos consecutivos.

No geral, a solução cumpre os objetivos definidos e demonstra a aplicação prática de várias primitivas de sistemas operativos, como `fork`, `execvp`, `pipe`, `dup2`, `open`, `read`, `write`, `FIFOs` e `waitpid`, no contexto de um sistema com múltiplos processos concorrentes.